

API Reference

Revised Page

Version Number	Revision (Content) Description	Revision Time
V1.0.0	C++ version launched, new document created. (Corresponding to SDK version number V3.0.0)	2026/04/30

1. Control Class DexH15Control

Responsible for core operations such as serial communication, motor control, and sensor reading of the dexterous hand.

1.1 connectModbusAuto

Function Overview

Automatically connect to the first dexterous hand device under the current serial port. Suitable for single device scenario, the system will automatically complete the device discovery and connection process.

Function Prototype

```
bool connectModbusAuto(const std::string& hand_port_num, int16_t& device_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
hand_port_num	const std::string&	Input	Serial port number connected to the dexterous hand (e.g., "/dev/ttyUSB0")
device_address	int16_t&	Output	The first device address returned after a successful connection

Return Value

- `true`: Connection successful
- `false`: Connection failed

Special Instructions

If an error occurs during the execution of the program, an exception will be thrown.

1.2 openModbusDevice

Function Overview

Open the specified serial port device and establish a Modbus communication connection.

Function Prototype

```
bool openModbusDevice(const std::string& hand_port_num, int32_t
baud_rate);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
hand_port_num	const std::string&	Input	Serial port number connected to the dexterous hand
baud_rate	int32_t	Input	Baud rate for serial device communication (4000000, 921600, 460800, 230400, 115200, 57600)

Return Value

- `true`: Open successful
- `false`: Failed to open

1.3 scanModbusDevices

Function Overview

Scan all communicable device addresses under the opened serial port.

Function Prototype

```
std::vector<int16_t> scanModbusDevices(const std::string& hand_port_num);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
hand_port_num	const std::string&	Input	Serial port number connected to the dexterous hand

Return Value

Returns the set of addresses of all communicable devices under the opened serial port (`std::vector<int16_t>`).

1.4 initModbusDevice

Function Overview

Initialize the Modbus device configuration of the dexterous hand.

Function Prototype

```
int initModbusDevice(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

- 1: Initialization successful
 - 0: Initialization failed
-

1.5 disconnectModbus

Function Overview

Close the current Modbus serial port connection and release the serial port resources.

Function Prototype

```
bool disconnectModbus();
```

Parameter Description

None

Return Value

- true: Disconnection successful
 - false: Disconnection failed
-

1.6 isModbusDeviceConnected

Function Overview

Check the Modbus connection status of the dexterous hand at the specified slave address.

Function Prototype

```
bool isModbusDeviceConnected(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

- true: Device is connected
- false: Device not connected

1.7 setModbusBaudrate

Function Overview

Set the communication baud rate of the Modbus slave station.

Function Prototype

```
bool setModbusBaudrate(int16_t slave_address, int32_t baud_rate);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
baud_rate	int32_t	Input	Target baud rate (4000000, 921600, 460800, 230400, 115200, 57600)

Return Value

- true: Setting successful
- false: Setting failed

Special Instructions

The baud rate setting must be completed before using the serial port, and after the setting is completed the device needs to be restarted to take effect. All devices connected to the same serial port must maintain the same baud rate.

1.8 getModbusBaudrate

Function Overview

Get the communication baud rate of the current Modbus slave station.

Function Prototype

```
int32_t getModbusBaudrate(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns the current baud rate value of the slave device (int32_t).

1.9 setModbusDeviceAddress

Function Overview

Modify the device address of the Modbus slave station.

Function Prototype

```
bool setModbusDeviceAddress(int16_t slave_address, int16_t new_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Current slave address (0x01 ~ 0xF7)
new_address	int16_t	Input	New slave address (0x01 ~ 0xF7)

Return Value

- true: Setting successful
- false: Setting failed

Special Instructions

The setting of the slave address must be completed before use. After the setting is completed, the device needs to be restarted to take effect.

1.10 openUsbDevice

Function Overview

Open the USB serial device and establish a connection, then automatically obtain the slave address.

Function Prototype

```
bool openUsbDevice(const std::string& hand_port_num, int16_t& device_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
hand_port_num	const std::string&	Input	USB Device Number
device_address	int16_t&	Output	Slave address returned after successful connection

Return Value

- `true`: Open successful
 - `false`: Open failed
-

1.11 disconnectUsb

Function Overview

Close the USB device connection and release related resources.

Function Prototype

```
bool disconnectUsb();
```

Parameter Description

None

Return Value

- `true`: Operation execution completed
-

1.12 setMotorControlMode

Function Overview

Set the control mode of the motor. The default mode is position mode, which can be switched to current mode or speed mode as needed.

Function Prototype

```
bool setMotorControlMode(int16_t slave_address, const ControlMode mode);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	<code>int16_t</code>	Input	Slave Address
mode	<code>const ControlMode</code>	Input	Motor control mode enumeration value

Explanation of ControlMode Enumeration Values

Enumerated Value	Value	Description
CURRENT_CONTROL_MODE	1	Current Loop Control Mode
VELOCITY_CONTROL_MODE	2	Speed loop control mode
POSITION_CONTROL_MODE	3	Position Loop Control Mode (Default)

Return Value

- `true`: Setting successful
 - `false`: Setting failed
-

1.13 getHandType

Function Overview

Query the left or right hand type of the dexterous hand.

Function Prototype

```
HandType getHandType(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns `HandType` enum value, indicating the left or right hand type of the dexterous hand.

Enumerated Value	Value	Description
NONE_HAND	0	Invalid dexterous hand type
LEFT_HAND	1	Left Hand
RIGHT_HAND	2	Right Hand

1.14 setMotorMaxCurrent

Function Overview

Set the maximum current for all motors to limit the peak grip force.

Function Prototype

```
bool setMotorMaxCurrent(int16_t slave_address, const Dex15MotorCurrent& max_current);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
max_current	const Dex15MotorCurrent&	Input	Maximum current configuration for each motor, with a value range of [0, 1000]

Return Value

- `true`: Setting successful
- `false`: Setting failed

1.15 getMotorMaxCurrentConfig

Function Overview

Get the maximum current configuration of all motors in the current slave device.

Function Prototype

```
Dex15MotorCurrent getMotorMaxCurrentConfig(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns the `Dex15MotorCurrent` structure, which contains the maximum current configuration for all motors.

1.16 setMotorMaxVelocity

Function Overview

Set the maximum speed of all motors.

Function Prototype

```
bool setMotorMaxVelocity(int16_t slave_address, const Dex15MotorSpeed& max_speed);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
max_speed	const Dex15MotorSpeed&	Input	Maximum speed configuration for each motor, with a value range of [0, 1000]

Return Value

- `true`: Setting successful
 - `false`: Setting failed
-

1.17 getMotorMaxVelocityConfig

Function Overview

Get the maximum speed configuration of all motors of the current slave device.

Function Prototype

```
Dex15MotorSpeed getMotorMaxVelocityConfig(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns **Dex15MotorSpeed** structure, which contains the maximum speed configuration for all motors.

1.18 enableMotor

Function Overview

Enable (power on and activate) all motors of the specified slave station.

Function Prototype

```
bool enableMotor(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

- **true**: Enable success
 - **false**: Enable failure
-

1.19 disableMotor

Function Overview

Disable (power off) all motors of the specified slave station.

Function Prototype

```
bool disableMotor(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

- `true`: Lower enable successful
- `false`: Lower enable failure

1.20 isMotorEnabled

Function Overview

Query the current enable status of the specified slave motor.

Function Prototype

```
bool isMotorEnabled(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

- `true`: Motor is enabled
- `false`: Motor is not enabled

1.21 setJointPositionsAngle (All Joints)

Function Overview

Control the target angles of all 11 finger joints (normalized angles).

Function Prototype

```
int setJointPositionsAngle(int16_t slave_address, const  
std::vector<double>& joint_angles);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	<code>int16_t</code>	Input	Slave Address
joint_angles	<code>const std::vector<double>&</code>	Input	Set of normalized angle values for 11 adjustable joints

Description of joint_angles Elements

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	Corresponds to the Finger	Value Range	Description
0	index_joint1	index finger	[0, 1000]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 1000]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 1000]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 1000]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 1000]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 1000]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 1000]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 1000]	End joint, bending and extension
8	thumb_joint1	Thumb	[-500, 500]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 1000]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 1000]	End joint, bending and extension

Return Value

- `1`: Setup successful
- `0`: Setting failed

1.22 setJointPositionsAngle (Specify Finger)

Function Overview

Set the normalized joint angle of a specified single finger.

Function Prototype

```
int setJointPositionsAngle(int16_t slave_address, FingerType finger_id,
const std::vector<double>& joint_angles);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
finger_id	FingerType	Input	Enumeration value of the target finger
joint_angles	const std::vector<double>&	Input	Set of normalized angle values for the specified finger

Definition of FingerType Enum Values

Enumerated Value	Value	Description
INVALID_FINGER	0	Invalid value
INDEX_FINGER	1	index finger
MIDDLE_FINGER	2	middle finger
RING_FINGER	3	ring finger
PINKY_FINGER	4	Little Finger
THUMB_FINGER	5	Thumb

Return Value

- 1: Setup successful
- 0: Setting failed

1.23 calculateRealAngle (Control Class)

Function Overview

Calculate the corresponding real physical angle (unit: degrees) based on the normalized angle value.

Function Prototype

```
int calculateRealAngle(int16_t slave_address,
                      const std::vector<double>& normalize_angle,
                      std::vector<double>& real_angle);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
normalize_angle	const std::vector<double>&	Input	Set of normalized angle values for 11 adjustable joints
real_angle	std::vector<double>&	Output	The set of 11 calculated real angle values of joints

Description of normalize_angle Elements

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	Corresponds to the Finger	Value Range	Description
0	index_joint1	index finger	[0, 1000]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 1000]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 1000]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 1000]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 1000]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 1000]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 1000]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 1000]	End joint, bending and extension
8	thumb_joint1	Thumb	[-500, 500]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 1000]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 1000]	End joint, bending and extension

Return Value

- 1: Calculation successful
- 0: Calculation failed

1.24 calculateNormalizeAngle (Control Class)

Function Overview

Calculate the corresponding normalized angle based on the real physical angle value.

Function Prototype

```
int calculateNormalizeAngle(int16_t slave_address,
                           const std::vector<double>& real_angle,
                           std::vector<double>& normalize_angle);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
real_angle	const std::vector<double>&	Input	Set of real angle values (unit: degrees) for 11 adjustable joints
normalize_angle	std::vector<double>&	Output	The calculated set of normalized angle values for 11 adjustable joints

Description of real_angle Elements

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	Corresponds to the Finger	Value Range	Description
0	index_joint1	index finger	[0, 80]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 80]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 80]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 80]	End joint, bending and extension

Index	Joint Identification	Corresponds to the Finger	Value Range	Description
4	ring_joint1	ring finger	[0, 80]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 80]	End joint, bending and extension
6	little_joint1	Little finger	[0, 80]	Proximal joint, flexion and extension
7	little_joint2	Little finger	[0, 80]	End joint, bending and extension
8	thumb_joint1	Thumb	[0, 109.6]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 87]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 120.1]	End joint, bending and extension

Return Value

- 1: Calculation successful
- 0: Calculation failed

1.25 setMotorTargetCurrent (All Motors)

Function Overview

In current mode, set the target current for all motors. The larger the target current value, the stronger the hand grip force.

Function Prototype

```
int setMotorTargetCurrent(int16_t slave_address, int16_t target_current);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
target_current	int16_t	Input	Unified target current for all motors, with a value range of [-1000, 1000]

Return Value

- 1: Setup successful

- 0: Setting failed

1.26 setMotorTargetCurrent (Specified Motor)

Function Overview

In current mode, set the target current for the specified single motor.

Function Prototype

```
int setMotorTargetCurrent(int16_t slave_address,  
                          FingerMotor motor_id,  
                          int16_t target_current);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
motor_id	FingerMotor	Input	Enumerated index value of the target motor
target_current	int16_t	Input	Target current value, range [-1000, 1000]

Definition of FingerMotor Enumeration Values

Enumerated Value	Value	Description
INVALID_MOTOR	0	Invalid value
INDEX_MOTOR	1	Index finger motor
MIDDLE_MOTOR	2	Middle finger motor
RING_MOTOR	3	Ring finger motor
PINKY_MOTOR	4	Little Finger Motor
THUMB_MOTOR_1	5	Thumb side swing motor
THUMB_MOTOR_2	6	Thumb Motor 2
THUMB_MOTOR_3	7	Thumb Motor 3

Return Value

- 1: Setup successful
- 0: Setting failed

1.27 getMotorActualCurrent

Function Overview

Get the actual current values of all current motors.

Function Prototype

```
int getMotorActualCurrent(int16_t slave_address, Dex15MotorCurrent& actual_current);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
actual_current	Dex15MotorCurrent&	Output	Contains the actual current of all motors, ranging from [-1000, 1000]

Return Value

- 1: Acquisition Successful
- 0: Acquisition failed

1.28 setMotorTargetVelocity (All Motors)

Function Overview

Under Speed Mode, the target speed of all motors is uniformly set to control the overall movement speed of the hand.

Function Prototype

```
int setMotorTargetVelocity(int16_t slave_address, int16_t target_speed);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
target_speed	int16_t	Input	The unified target speed for all 13 motors, with a value range of [-1000, 1000]

Return Value

- 1: Setup successful
- 0: Setting failed

1.29 setMotorTargetVelocity (Specify Motor)

Function Overview

Under speed control mode, set the target speed of the specified single motor.

Function Prototype

```
int setMotorTargetVelocity(int16_t slave_address, FingerMotor motor_id,
int16_t target_speed);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
motor_id	FingerMotor	Input	Enumerated index value of the target motor
target_speed	int16_t	Input	Target speed value, range [-1000, 1000]

Definition of FingerMotor Enumeration Values

Enumerated Value	Value	Description
INVALID_MOTOR	0	Invalid value
INDEX_MOTOR	1	Index finger motor
MIDDLE_MOTOR	2	Middle finger motor
RING_MOTOR	3	Ring finger motor
PINKY_MOTOR	4	Little Finger Motor
THUMB_MOTOR_1	5	Thumb side swing motor
THUMB_MOTOR_2	6	Thumb Motor 2
THUMB_MOTOR_3	7	Thumb Motor 3

Return Value

- 1: Setup successful
- 0: Setting failed

1.30 getMotorActualVelocity

Function Overview

Get the actual operating speeds of all current motors.

Function Prototype

```
int getMotorActualVelocity(int16_t slave_address, Dex15MotorSpeed& actual_speed);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
actual_speed	Dex15MotorSpeed&	Output	Contains the actual speed of all motors, ranging from [-1000, 1000]

Return Value

- 1: Success in Acquisition
 - 0: Acquisition failed
-

1.31 calibrateSensor

Function Overview

Calibration of tactile sensors.

Function Prototype

```
bool calibrateSensor(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

- true: Calibration successful
 - false: Calibration failed
-

1.32 getJointPositionsAngle (All Joints)

Function Overview

Get the current normalized angles of all 11 finger joints.

Function Prototype

```
int getJointPositionsAngle(int16_t slave_address, std::vector<double>&
joint_angles);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
joint_angles	std::vector<double>&	Output	Set of current normalized angle values for 11 joints

Description of joint_angles Elements

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	Corresponds to the Finger	Value Range	Description
0	index_joint1	index finger	[0, 1000]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 1000]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 1000]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 1000]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 1000]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 1000]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 1000]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 1000]	End joint, bending and extension
8	thumb_joint1	Thumb	[-500, 500]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 1000]	Proximal joint, flexion and extension

Index	Joint Identification	Corresponds to the Finger	Value Range	Description
10	thumb_joint3	Thumb	[0, 1000]	End joint, bending and extension

Return Value

- 1: Acquisition Successful
- 0: Acquisition failed

1.33 getJointPositionsAngle (Specified Finger)

Function Overview

Get the current joint angle of the specified single finger.

Function Prototype

```
int getJointPositionsAngle(int16_t slave_address,
                          FingerType finger_id,
                          std::vector<double>& joint_angles);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
finger_id	FingerType	Input	Enumerated index value of the target finger
joint_angles	std::vector<double>&	Output	Set of joint angle values for the specified finger

Definition of FingerType Enum Values

Enumerated Value	Value	Description
INVALID_FINGER	0	Invalid value
INDEX_FINGER	1	index finger
MIDDLE_FINGER	2	middle finger
RING_FINGER	3	ring finger
PINKY_FINGER	4	Little Finger
THUMB_FINGER	5	Thumb

Return Value

- 1: Acquisition Successful
- 0: Acquisition failed

1.34 getMotorControlMode

Function Overview

Get the control mode of the current motor.

Function Prototype

```
ControlMode getMotorControlMode(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns **ControlMode** enum value:

Enumerated Value	Value	Description
NONE	0	Invalid value (only appears when retrieval fails)
CURRENT_CONTROL_MODE	1	Current Loop Control Mode
VELOCITY_CONTROL_MODE	2	Speed loop control mode
POSITION_CONTROL_MODE	3	Position loop control mode

1.35 getFingerTactile

Function Overview

Obtain the resultant force information of all finger tactile sensors. The hand has a total of 5 sensors, each returning a three-axis resultant force.

Function Prototype

```
std::vector<ForcePoint> getFingerTactile(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns a list containing 5 **ForcePoint** instances, each **ForcePoint** containing:

Field	Type	Description
x	float	Sensor X-axis force component, unit 0.1N
y	float	Sensor Y-axis force component, unit 0.1N
z	float	Sensor Z-axis force component, unit 0.1N

1.36 getFingerTactileDetail

Function Overview

Obtain the distributed force detailed information of all finger tactile sensors. The hand has a total of 5 sensors, with a total of 335 distributed force points.

Function Prototype

```
std::vector<ForcePoint> getFingerTactileDetail(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns a list containing 327 **ForcePoint** instances. The distribution rules are as follows:

Finger	Force Points
index finger	50
middle finger	50
ring finger	50
Little Finger	50
Thumb	127

Each **ForcePoint** contains x, y, z three-axis components, with a unit of 0.1N.

1.37 getTemperature

Function Overview

Obtain the temperature data of all finger sensors. The hand has a total of 5 sensors, returning 5 temperature readings.

Function Prototype

```
std::vector<FingerTemp> getTemperature(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns a list containing 5 `FingerTemp` instances, each `FingerTemp` containing:

Field	Type	Description
finger_name	std::string	Finger Name
sensor_ID	int	Sensor ID
temperature	float	Sensor Temperature Value (1°C)

1.38 getFingerTactileData

Function Overview

Obtain in one go the resultant force, distributed force, and temperature data of the specified single finger.

Function Prototype

```
void getFingerTactileData(int16_t slave_address,  
                          FingerType finger_id,  
                          ForcePoint& finger_tac,  
                          std::vector<ForcePoint>& finger_tac_detail,  
                          FingerTemp& temperature);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
finger_id	FingerType	Input	Enumeration type value of the target finger
finger_tac	ForcePoint&	Output	Resultant force information of the tactile sensors of this finger
finger_tac_detail	std::vector<ForcePoint>&	Output	Distribution force information of the tactile sensors on this finger
temperature	FingerTemp&	Output	The sensor temperature data of this finger

Definition of FingerType Enum Values

Enumerated Value	Value	Description
INVALID_FINGER	0	Invalid value
INDEX_FINGER	1	index finger
MIDDLE_FINGER	2	middle finger
RING_FINGER	3	ring finger
PINKY_FINGER	4	Little Finger
THUMB_FINGER	5	Thumb

Return Value

No return value (`void`).

1.39 getAllTactileData

Function Overview

Obtain all at once the resultant force, distributed force, and temperature data of all fingers.

Function Prototype

```
void getAllTactileData(int16_t slave_address,
                      std::vector<ForcePoint>& finger_tac,
                      std::vector<ForcePoint>& finger_tac_detail,
                      std::vector<FingerTemp>& temperature);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
finger_tac	std::vector<ForcePoint>&	Output	Force information of the tactile sensors of all fingers combined
finger_tac_detail	std::vector<ForcePoint>&	Output	Tactile sensor distributed force information for all fingers
temperature	std::vector<FingerTemp>&	Output	Sensor temperature data for all fingers

Return Value

No return value (`void`).

1.40 checkHandStatus

Function Overview

Check the overall operating status of the dexterous hand.

Function Prototype

```
int checkHandStatus(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Value	Description
1	Normal operation
2	Fault-tolerant operation state (an error has occurred, but it does not affect the use of core interfaces)
3	Non-operational state

1.41 getFaultCode

Function Overview

Obtain the fault code of the current motor to diagnose the cause of motor failure.

Function Prototype

```
std::vector<FingerMotorsError> getFaultCode(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns `std::vector<FingerMotorsError>` fault code list, containing the current fault information for each motor.

Error Name	Error Code	Error Description	Explanation	Solution
OVER_VOLT	0	Finger motor overvoltage	The drive supply voltage is higher than 15V and lasts for 2 seconds	After the voltage is restored, simply execute the clear error command
UNDER_VOLT	1	Finger motor under voltage	The drive supply voltage is below 10V and lasts for 2 seconds	After the voltage is restored, simply execute the clear error command
OVER_CURRENTU	2	Finger motor U-phase overcurrent	U-phase current is too large	After the current returns to normal, simply execute the error clearing command
OVER_CURRENTY	3	Finger motor V-phase overcurrent	Excessive V-phase current	After the current returns to normal, simply execute the error clearing command
OVER_CURRENTW	4	Finger motor W-phase overcurrent	W-phase current is too large	After the current returns to normal, simply execute the

Error Name	Error Code	Error Description	Explanation	Solution
				error clearing command
OVER_TEMPERATURE	5	Finger motor overheating	Motor temperature is too high	After waiting for the finger motor temperature to return to the normal range, simply execute the error clearing command
ADC_CALIB	6	Finger motor hardware self-check error	AD Static Deviation +10%	1. Check the magnetic field environment of the driver attachment. After eliminating interference, power off and restart. 2. If the issue still cannot be resolved, replace the driver board.
FOC_ERR	7	Finger motor FOC calculation error	None	None
OPD_IQ	8	Finger motor IQ overcurrent	Motor IQ current is too high	After the overcurrent of the IQ current is eliminated, simply execute the error clearing operation
OPD_U	9	Finger motor U-phase open circuit	Detection performed during power-on initialization	Check whether the U-phase wire of the motor is disconnected. After restoration, execute the error clearing operation
OPD_V	10	Phase V open circuit fault in finger motor	Detection performed during power-on initialization	Check whether the V-phase wire of the motor is disconnected. After restoration, execute

Error Name	Error Code	Error Description	Explanation	Solution
				the error clearing operation
OPD_W	11	Finger motor W-phase open circuit	Detection performed during power-on initialization	Check whether the W-phase wire of the motor is disconnected. After restoration, execute the error clearing operation
POS_REF_OVER	12	Finger motor position exceeds upper limit	None	None
POS_REF_UNDER	13	Finger motor position is below the lower limit	None	None
ABZ_ERR	14	Finger motor encoder anomaly	Magnetic encoding cycle loss count greater than 0.25%	The encoder at the motor end has lost data. Check the encoder. After powering off and restarting, observe the phenomenon. If the error persists, return to factory.
ABZ_Z_ERR	15	Finger motor encoder Z-phase anomaly	Z phase cannot be found during initialization	Encoder Z-phase error at the motor end. Check the encoder. After powering off and restarting, observe the phenomenon. If the error persists, return to factory.
ABZ_CALIB_ERR	16	Encoder calibration error	Encoder cable not properly connected or not calibrated	Troubleshoot the encoder or wiring

Error Name	Error Code	Error Description	Explanation	Solution
SPEED_REF_OVER	17	Given speed limit exceeded	None	None
CURRENT_REF_OVER	18	Given current exceeds limit	None	None
JOINT_ENC_DISCONNECT	19	Magnetic encoder disconnection on the electric drive board	None	None
NORM_POS_NEGA_WARN	29	Normalization motor stroke appears small negative number	Motor stroke value is less than 0 and greater than -20	None
NORM_POS_NEGA_ERR	30	Normalized motor stroke shows large negative number	Motor stroke value is less than -20	None
DISCONNECT	31	Finger motor not connected	Communication between the finger drive and the control board is disconnected	After connection, the system will automatically clear errors and recover

1.42 clearFaultCode

Function Overview

Clear the fault code of the current motor.

Function Prototype

```
bool clearFaultCode(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

- `true`: Clear successful
- `false`: Clear failed

Special Instructions

Not all fault codes can be cleared. For details, please refer to the SDK's accompanying documentation.

1.43 getFirmwareVersion

Function Overview

Obtain the firmware version information of the device.

Function Prototype

```
const FirmwareVersion getFirmwareVersion(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	<code>int16_t</code>	Input	Slave Address

Return Value

Returns the `FirmwareVersion` structure, which contains detailed information about the device's firmware version.

1.44 getSDKVersion

Function Overview

Get the version number of the current SDK.

Function Prototype

```
const std::string getSDKVersion();
```

Parameter Description

None

Return Value

Returns the SDK version number string (`std::string`).

1.45 initMotorPosition

Function Overview

Initialize the positions of all motors to return the dexterous hand to its initial zero position state.

Function Prototype

```
int initMotorPosition(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

- 1: Initialization successful
- 0: Initialization failed

Special Instructions

It is recommended to call immediately after the device is powered on, with a maximum execution time of approximately 5 seconds.

1.46 setMotorTargetPosition (All Motors)

Function Overview

Unifiedly set the target positions of all motors through the motor position parameter structure.

Function Prototype

```
int setMotorTargetPosition(int16_t slave_address,  
                           const Dex15MotorPosition& target_position);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
target_position	const Dex15MotorPosition&	Input	Target position values of all motors

Return Value

- 1: Setup successful
- 0: Setting failed

1.47 setMotorTargetPosition (Specify Motor)

Function Overview

Set the target position of the specified single motor.

Function Prototype

```
int setMotorTargetPosition(int16_t slave_address,
                           FingerMotor motor_id,
                           int16_t target_position);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
motor_id	FingerMotor	Input	Enumerated index value of the target motor
target_position	int16_t	Input	Target motor position value

Definition of FingerMotor Enumeration Values

Enumerated Value	Value	Description
INVALID_MOTOR	0	Invalid value
INDEX_MOTOR	1	Index finger motor
MIDDLE_MOTOR	2	Middle finger motor
RING_MOTOR	3	Ring finger motor
PINKY_MOTOR	4	Little Finger Motor
THUMB_MOTOR_1	5	Thumb side swing motor
THUMB_MOTOR_2	6	Thumb Motor 2
THUMB_MOTOR_3	7	Thumb Motor 3

Return Value

- 1: Setup successful
- 0: Setting failed

1.48 getMotorPosition

Function Overview

Get the position information of all current motors.

Function Prototype

```
const Dex15MotorPosition getMotorPosition(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns `Dex15MotorPosition` structure, which contains the current position information of all motors.

1.49 getPressPositionForce

Function Overview

Get the set of position force points of all current finger pressing forces, with the unit being 0.1N.

Function Prototype

```
std::vector<PressForcePoint> getPressPositionForce(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns `std::vector<PressForcePoint>`, containing the press force position point data for all fingers, with the unit of 0.1N.

1.50 getDeviceSN

Function Overview

Obtain the SN serial number of the device.

Function Prototype

```
std::string getDeviceSN(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns the device SN code string.

1.51 getHardwareVersion

Function Overview

Get the hardware version information of the device.

Function Prototype

```
HardwareVersion getHardwareVersion(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns **HardwareVersion** structure, which contains detailed information about the device's hardware version.

1.52 getModbusConfig

Function Overview

Obtain the Modbus communication configuration information of the current slave device.

Function Prototype

```
ModbusConfig getModbusConfig(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns `ModbusConfig` structure, which contains the current Modbus communication configuration.

1.53 getDeviceInfo

Function Overview

Unify the acquisition of all device information, such as the SN code, hardware version, firmware version, and Modbus configuration, of the slave device through a single call.

Function Prototype

```
bool getDeviceInfo(int16_t slave_address,
                  std::string& sn,
                  HardwareVersion& hardware_version,
                  FirmwareVersion& firmware_version,
                  ModbusConfig& modbus_config);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
sn	std::string&	Output	Device SN Code
hardware_version	HardwareVersion&	Output	Device hardware version information
firmware_version	FirmwareVersion&	Output	Device firmware version information
modbus_config	ModbusConfig&	Output	Device Modbus communication configuration information

Return Value

- `true`: Acquisition successful
 - `false`: Acquisition failed
-

1.54 getJointMagneticEncoder

Function Overview

Obtain the raw data of the magnetic encoders of all joints of the current slave device.

Function Prototype

```
std::vector<int16_t> getJointMagneticEncoder(int16_t slave_address);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Return Value

Returns a list containing the values of all joint magnetic encoders (`std::vector<int16_t>`).

1.55 getActionPosition

Function Overview

Load and read the motor position information corresponding to the built-in or custom action sequence.

Function Prototype

```
int getActionPosition(int16_t slave_address,  
                      int16_t index,  
                      Dex15MotorPosition& action_position,  
                      bool is_customized = true);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Action Sequence Number (range [1, 20] when is_customized=true; range [1, 10] when is_customized=false)
action_position	Dex15MotorPosition&	Output	Acquired motor position data
is_customized	bool	Input	true: Custom action; false: Official built-in action (default true)

Return Value

- 1: Success in Acquisition

- 0: Acquisition failed

1.56 runAction

Function Overview

Execute the built-in or custom action with the specified number.

Function Prototype

```
int runAction(int16_t slave_address, int16_t index, bool is_customized = true);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Action Sequence Number (range [1, 20] when is_customized=true; range [1, 10] when is_customized=false)
is_customized	bool	Input	true: Custom action; false: Official built-in action (default true)

Return Value

- 1: Execution successful
- 0: Execution failed

1.57 saveCustomizedAction (Specify Action Frame)

Function Overview

Save the specified motor position data as a user-defined action.

Function Prototype

```
int saveCustomizedAction(int16_t slave_address,
                        int16_t index,
                        const Dex15MotorPosition& action);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Custom action sequence number, range [1, 20]
action	const Dex15MotorPosition&	Input	Motor position data to be saved

Return Value

- 1: Save Successful
- 0: Save failed

1.58 saveCustomizedAction (Current Position)

Function Overview

Save the current real-time position of the dexterous hand directly as a custom action with the specified number.

Function Prototype

```
int saveCustomizedAction(int16_t slave_address, int16_t index);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Custom action sequence number, range [1, 20]

Return Value

- 1: Save Successful
- 0: Save failed

1.59 saveCustomizedAction (Joint Angle)

Function Overview

Calculate the motor position based on the specified set of joint normalized angles, and then save it as a custom action.

Function Prototype


```
int saveCustomizedAction(int16_t slave_address,
                        int16_t index,
                        std::vector<double> joint_angle_list);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
index	int16_t	Input	Custom action sequence number, range [1, 20]
joint_angle_list	std::vector<double>	Input	Set of joint normalized angles to be saved (11 adjustable joints)

Return Value

- 1: Save Successful
- 0: Save failed

1.60 setCommonForceCtlParam

Function Overview

Configure the initial position and control parameters of the common force control. After the configuration is completed, calculate the target command for each motion through `computeCommonForceTargetCommand`.

Function Prototype

```
int setCommonForceCtlParam(int16_t slave_address,
                           const std::vector<double>& joint_angles,
                           const ForceCtlParam& param);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address

Parameter Name	Type	Input/Output	Description
joint_angles	<code>const std::vector<double>&</code>	Input	The initial position of the dexterous hand during normal force control requires the input of the normalized angle values of 11 adjustable joints
param	<code>const ForceCtlParam&</code>	Input	Force control parameter configuration, default parameters will be used if not set

Return Value

- `1`: Configuration successful
- `0`: Configuration failed

Special Instructions

- The parameter configuration of normal force control and tangential force control will overwrite each other, and the two cannot be set and used simultaneously. After being overwritten, the parameters need to be reset before the calculation of the target position can be performed.
- When the current position of the dexterous hand is different from the initial position `joint_angles` where force control needs to be performed, it is necessary to wait for the dexterous hand to reach the initial position by sleeping as appropriate.

1.61 computeCommonForceTargetCommand

Function Overview

In normal force control mode, calculate the target joint angle command for the next motion based on the current force sensor data.

Function Prototype

```
int computeCommonForceTargetCommand(int16_t slave_address,
std::vector<double>& command);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	<code>int16_t</code>	Input	Slave Address
command	<code>std::vector<double>&</code>	Output	Normalized angle command for the next movement, containing 11 adjustable joint data

Return Value

- 1: Calculation successful
- 0: Calculation failed

1.62 setTangentialForceCtlParam

Function Overview

Configure the initial position and control parameters of tangential force control. After configuration is completed, calculate the target command for each motion through `computeTangentialForceTargetCommand`.

Function Prototype

```
int setTangentialForceCtlParam(int16_t slave_address,  
                               const std::vector<double>& joint_angles,  
                               const TanForceCtlParam& param);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
joint_angles	const std::vector<double>&	Input	The initial position of the dexterous hand during tangential force control requires the input of the normalized angle values of 11 adjustable joints
param	const TanForceCtlParam&	Input	Tangential force control parameter configuration, default parameters will be used if not set

Return Value

- 1: Configuration successful
- 0: Configuration failed

Special Instructions

- The parameter configuration of normal force control and tangential force control will overwrite each other, and the two cannot be set and used simultaneously. After being overwritten, the parameters need to be reset before the calculation of the target position can be performed.
- When the current position of the dexterous hand is different from the initial position `joint_angles` where force control needs to be performed, it is necessary to wait for the

dexterous hand to reach the initial position by sleeping as appropriate.

1.63 computeTangentialForceTargetCommand

Function Overview

In the tangential force control mode, calculate the target joint angle command for the next motion based on the current force sensor data.

Function Prototype

```
int computeTangentialForceTargetCommand(int16_t slave_address,  
                                         std::vector<double>& command);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
slave_address	int16_t	Input	Slave Address
command	std::vector<double>&	Output	The normalized angle command for the next movement, containing 11 adjustable joint data

Return Value

- 1: Calculation successful
 - 0: Calculation failed
-

2. Camera Control Class DexH15Camera

Responsible for the connection, image acquisition, and parameter configuration of the camera equipment supporting the dexterous hand.

2.1 connectCameraDevice

Function Overview

Open and connect to the specified camera device.

Function Prototype

```
void connectCameraDevice(const std::string& camera_port_num);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
camera_port_num	<code>const std::string&</code>	Input	Camera Device Number

Return Value

No return value (`void`). Throws an exception when the connection fails.

2.2 releaseCameraDevice

Function Overview

Release camera device resources and disconnect.

Function Prototype

```
void releaseCameraDevice();
```

Parameter Description

None

Return Value

No return value (`void`).

2.3 getFrame

Function Overview

Capture the current frame image from the camera.

Function Prototype

```
cv::Mat getFrame();
```

Parameter Description

None

Return Value

Returns an image frame of type `cv::Mat`, with a default resolution of 640×480 and a frame rate of 30 frames per second. If the camera is not opened, it returns an empty Mat.

2.4 showImage

Function Overview

Displays the specified image frame in the window.

Function Prototype

```
void showImage(const std::string& imgName, const cv::Mat& img);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
imgName	const std::string&	Input	Display the name of the window
img	const cv::Mat&	Input	Image to be displayed

Return Value

No return value (void).

2.5 saveImage

Function Overview

Save the specified image frame to a local file.

Function Prototype

```
void saveImage(const std::string& imgPath, const cv::Mat& img);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
imgPath	const std::string&	Input	File path for image saving
img	const cv::Mat&	Input	Image to be saved

Return Value

No return value (void).

2.6 setCameraConfig

Function Overview

Set the resolution and frame rate parameters of the camera.

Function Prototype

```
bool setCameraConfig(int width, int height, float fps);
```

Parameter Description

Parameter Name	Type	Input/Output	Description
width	int	Input	Image resolution width (pixels)
height	int	Input	Image resolution height (pixels)
fps	float	Input	Camera frame rate (frames per second)

Return Value

- `true`: Configuration successful
- `false`: Configuration failed

Camera Parameter Description

Set camera parameters, including resolution and frame rate. Only 640×480, 1280×720, and 1920×1080 resolutions are supported. The frame rate only supports 30fps for 640×480 and 1280×720 resolutions, and 60fps for 1920×1080 resolution.

3. Kinematic Analysis Class DexH15Kinematic

Responsible for the forward and inverse kinematic transformation between the dexterous hand joint angles and motor positions, as well as the mutual conversion between normalized angles and real angles.

3.1 getSDKVersion

Function Overview

Get the version number of the current SDK.

Function Prototype

```
const std::string getSDKVersion();
```

Parameter Description

None

Return Value

Returns the SDK version number string (`std::string`).

3.2 inverseKinematicParse

Function Overview

Inverse Kinematics Analysis: Input the real angle values of 11 adjustable joints, calculate and output the corresponding 7 motor position values.

Function Prototype

```
int inverseKinematicParse(const HandType hand_type,
                          const std::vector<double>& real_angle,
                          std::vector<int16_t>& motor_pos,
                          const std::string& hardware_version = "");
```

Parameter Description

Parameter Name	Type	Input/Output	Description
hand_type	<code>const HandType</code>	Input	Left or right hand type
real_angle	<code>const std::vector<double>&</code>	Input	Set of real angle values for 11 adjustable joints (unit: °)
motor_pos	<code>std::vector<int16_t>&</code>	Output	7 motor position information obtained through parsing
hardware_version	<code>const std::string&</code>	Input	Hardware version (optional, default is empty)

Description of real_angle Elements

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	Corresponds to the Finger	True Angle Range (°)	Description
0	index_joint1	index finger	[0, 80]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 80]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 80]	Proximal joint, flexion and extension

Index	Joint Identification	Corresponds to the Finger	True Angle Range (°)	Description
3	middle_joint2	middle finger	[0, 80]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 80]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 80]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 80]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 80]	End joint, bending and extension
8	thumb_joint1	Thumb	[0, 109.6]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 87]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 120.1]	End joint, bending and extension

Return Value

- 1: Parsing successful
- 0: Parsing failed

3.3 forwardKinematicParse

Function Overview

Forward kinematics analysis: Input 7 motor position values and 8 magnetic encoder data, calculate and output the real angle values of 11 adjustable joints.

Function Prototype

```
int forwardKinematicParse(const HandType hand_type,
                          const std::vector<int16_t>& motor_pos,
                          const std::vector<int16_t>& magnetic_encoder,
                          std::vector<double>& real_angle,
                          const std::string& hardware_version = "");
```

Parameter Description

Parameter Name	Type	Input/Output	Description
hand_type	<code>const HandType</code>	Input	Left or right hand type
motor_pos	<code>const std::vector<int16_t>&</code>	Input	Set of 7 motor position values, with a range of [0, 10000]
magnetic_encoder	<code>const std::vector<int16_t>&</code>	Input	8 sets of magnetic encoder data
real_angle	<code>std::vector<double>&</code>	Output	The 11 real angle values of the joints obtained by parsing (unit: °)
hardware_version	<code>const std::string&</code>	Input	Hardware version (optional, default is empty)

Return Value

- `1`: Parsing successful
- `0`: Parsing failed

3.4 calculateRealAngle (Kinematics Class)

Function Overview

Calculate the corresponding real physical angles (unit: degrees) based on the normalized angle set of 11 adjustable joints.

Function Prototype

```
int calculateRealAngle(const std::vector<double>& normalize_angle,
                      std::vector<double>& real_angle,
                      const std::string& hardware_version = "");
```

Parameter Description

Parameter Name	Type	Input/Output	Description
normalize_angle	<code>const std::vector<double>&</code>	Input	Normalized angle set of 11 joints
real_angle	<code>std::vector<double>&</code>	Output	The calculated set of 11 real angles (unit: °)

Parameter Name	Type	Input/Output	Description
hardware_version	<code>const std::string&</code>	Input	Hardware version (optional, default is empty)

Description of normalize_angle Elements

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	Corresponds to the Finger	Normalized Input Range	Description
0	index_joint1	index finger	[0, 1000]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 1000]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 1000]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 1000]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 1000]	Proximal joint, flexion and extension
5	ring_joint2	ring finger	[0, 1000]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 1000]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 1000]	End joint, bending and extension
8	thumb_joint1	Thumb	[-500, 500]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 1000]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 1000]	End joint, bending and extension

Return Value

- `1`: Calculation successful
- `0`: Calculation failed

3.5 calculateNormalizeAngle (Kinematics Class)

Function Overview

Calculate the corresponding set of normalized angles based on the real physical angles of the 11 adjustable joints.

Function Prototype

```
int calculateNormalizeAngle(const std::vector<double>& real_angle,
                           std::vector<double>& normalize_angle,
                           const std::string& hardware_version = "");
```

Parameter Description

Parameter Name	Type	Input/Output	Description
real_angle	const std::vector<double>&	Input	Set of real angles of 11 joints (unit: °)
normalize_angle	std::vector<double>&	Output	The set of 11 normalized angles obtained from the calculation
hardware_version	const std::string&	Input	Hardware version (optional, default is empty)

Description of real_angle Elements

The order is as follows: index finger (2) → middle finger (2) → ring finger (2) → little finger (2) → thumb (3).

Index	Joint Identification	Corresponds to the Finger	True Angle Range (°)	Description
0	index_joint1	index finger	[0, 80]	Proximal joint, flexion and extension
1	index_joint2	index finger	[0, 80]	End joint, bending and extension
2	middle_joint1	middle finger	[0, 80]	Proximal joint, flexion and extension
3	middle_joint2	middle finger	[0, 80]	End joint, bending and extension
4	ring_joint1	ring finger	[0, 80]	Proximal joint, flexion and extension

Index	Joint Identification	Corresponds to the Finger	True Angle Range (°)	Description
5	ring_joint2	ring finger	[0, 80]	End joint, bending and extension
6	little_joint1	Little Finger	[0, 80]	Proximal joint, flexion and extension
7	little_joint2	Little Finger	[0, 80]	End joint, bending and extension
8	thumb_joint1	Thumb	[0, 109.6]	Metacarpophalangeal biaxial joint, lateral swing
9	thumb_joint2	Thumb	[0, 87]	Proximal joint, flexion and extension
10	thumb_joint3	Thumb	[0, 120.1]	End joint, bending and extension

Return Value

- 1: Calculation successful
- 0: Calculation failed